PROJECT OVERVIEW

SalesGenie AI

AI Sales Assistant & Lead Intelligence Platform

A clear guide to the project purpose, technology, backend, frontend, data, AI features and demo workflow.

Purpose: help a sales team organize prospective customers, understand which leads need attention, and prepare a personalized first-contact email draft.

Prepared for: Project presentation / academic explanation

Student: _____

Course / Class: _____

Date: _____

This document describes the implemented project based on its source code and project documentation. Features identified as future work are clearly separated from implemented functionality.

1. Project at a glance

SalesGenie AI is a small web-based sales support application. It provides one place to store lead records, import prospect lists, review a transparent company/lead analysis, and create outreach email drafts. It is built as a Python application with a separate browser-based UI and API.

Area	What it does
Lead management	Create, search, filter, update and delete company/contact prospect records.
CSV import	Upload a prospect list; validate rows and report inserted, duplicate and rejected rows.
Lead intelligence	Show company profile information, data completeness, business needs, opportunities, insights and information gaps.
Qualification score	Calculate a transparent rule-based score from 0 to 100 with factor details. It is a prioritization aid, not a prediction of a sale.
Outreach drafts	Prepare and store a first-touch email draft with selected tone, length and communication goal.
Dashboard	Display summary counters and lead breakdowns by status and industry.

Who is it for?

The intended user is a sales representative or sales manager who needs a simple prospect workspace. The current interface is an internal operator tool; it is not a customer-facing public product.

Why it is useful

- Reduces spreadsheet-only prospect tracking by keeping leads in a structured database.
- Catches invalid and duplicate records during CSV import instead of silently accepting them.
- Makes the reasoning behind the qualification score visible to the user.
- Keeps generated outreach as a reviewable draft; the system does not send email automatically.

2. Technology stack

The project uses a Python-first architecture. Streamlit provides the interactive screens, while FastAPI exposes the backend operations as HTTP endpoints. SQLAlchemy connects application models to SQLite or PostgreSQL.

Layer / technology	Role in the project
Python 3.11+	Main programming language for UI, API, service logic and automated tests.
Streamlit	Python framework that renders the browser UI and its multi-page navigation. No separate JavaScript build is required.
FastAPI	Backend web framework for REST endpoints, request handling, validation integration and generated API documentation.
Uvicorn	ASGI server process that runs the FastAPI application.
Pydantic v2	Defines and validates API request/response data; helps keep input shapes consistent.
SQLAlchemy 2.0	ORM used to map Python models to relational database tables and manage database sessions.
SQLite / PostgreSQL	SQLite is the default local development database; PostgreSQL is supported for a shared/production-style database.
Versioned SQL migrations	Custom runner applies schema changes once and records which versions have run.
Requests / httpx	HTTP clients for communication between the UI and API and for provider-backed outreach.
pytest	Automated tests for API behavior, business logic, imports, migrations and performance checks.
Docker Compose (optional)	Can start a PostgreSQL database service for local use.

Important design choices

- **Simple UI delivery:** Streamlit was chosen for an internal tool that can be built quickly in Python.
- **Database portability:** the same application can use SQLite locally and PostgreSQL via configuration.
- **Explainable scoring:** the project has no historical labeled sales outcomes for training a reliable predictive model, so it uses a visible rule-based heuristic.
- **Provider flexibility:** outreach can call an OpenAI-compatible chat-completions service; without a key, a labeled offline mock draft keeps the workflow demoable.

3. Backend: how it works

The backend is the application?s business and data layer. It receives HTTP requests from the UI, validates inputs, applies business rules, works with the database and returns structured JSON responses.

Backend request flow

Step	Component	Responsibility
1	Streamlit UI	Sends an HTTP request, such as creating a lead or asking for insights.
2	FastAPI router	Selects the endpoint and receives the request parameters/body. Routers are kept relatively thin.
3	Pydantic schema	Checks and normalizes the data shape and field constraints.
4	Service layer	Runs core rules: lead CRUD, CSV validation, duplicate detection, scoring, analysis and outreach draft generation.
5	SQLAlchemy + database	Reads/writes lead and supporting records; foreign keys connect related records.
6	JSON response	Returns the result or a clear error response to the API client in the UI.

Main backend modules

- **main.py:** creates the FastAPI app, configures CORS, registers routers and exposes health information.
- **config.py:** reads settings such as database URL, API prefix and optional language-model credentials from environment variables / .env.
- **database.py:** creates the SQLAlchemy engine, sessions and shared model base.
- **routers/:** groups HTTP routes for leads, company analysis and outreach.
- **services/:** contains business logic, kept separate from HTTP handling for testability.
- **schemas/ and models/:** define validated API data and persistent database entities respectively.

Representative endpoints

Endpoint	Purpose
GET /health	Confirms that the API process responds.
GET /api/leads and POST /api/leads	List/search leads and create a new lead.
PUT /api/leads/{id} and DELETE /api/leads/{id}	Update or remove a lead record.
POST /api/leads/import	Upload and process CSV prospect data.
GET /api/leads/{id}/insights	Return analysis and qualification score for one lead.
POST /api/outreach/generate	Generate and store an outreach draft.

FastAPI also serves interactive endpoint documentation at **http://127.0.0.1:8000/docs**.

4. Frontend: what the user sees

The frontend is a Streamlit application. The user opens it in a browser, while Streamlit executes the Python page code on the local machine. The UI calls the backend using a small HTTP client module, frontend/api_client.py.

Pages and features

Screen	User-facing functionality
Home / Dashboard	Summary metrics such as total leads and scored leads, plus charts grouped by status and industry. Includes a quick-start workflow.
Leads	Upload a CSV; see import results; manually add a record; search/filter the list; edit or delete a selected lead.
Intelligence	Choose a lead and view company details, data completeness, score and score band, score factors, business needs, opportunities, key insights and data gaps.
Outreach	Choose a lead, select email tone/length/goal, enter the offered product/service, generate a draft, preview it and review stored drafts.

Communication between UI and backend

The UI does not directly access the database. It uses HTTP requests to the FastAPI backend. For example, when a user imports a CSV, the browser UI sends the file to the import endpoint; the backend validates and processes it, writes accepted records to the database, and returns a row-by-row summary for display.

The main dashboard and pages are implemented in frontend/app.py and frontend/pages/. The shared API client centralizes the backend URL, request timeouts and error handling.

To use the interface, both processes must be running: the FastAPI backend and the Streamlit frontend. Streamlit alone starts only the UI; it does not start the API.

5. Data, intelligence and AI features

Database records

The main implemented records are leads, lead scores, company insights and outreach campaigns/drafts. A lead can have analysis and score history, and can be associated with multiple saved outreach drafts. Related records use lead IDs as foreign keys. The schema is managed by versioned SQL migrations.

Table / record	What it stores
leads	Company and contact details, status, notes, source and timestamps.
lead_scores	Score from 0?100, band, factor breakdown, methodology and calculation time.
company_insights	Company summary and lists of needs, opportunities, key insights and information gaps.
outreach_campaigns	Email subject/body, lead reference, selected options, draft status and provider/mock label.

CSV validation and duplicate handling

- CSV rows are parsed and checked individually; invalid rows can be reported without hiding the other results.
- An exact normalized email match is the strongest duplicate signal.
- A company plus contact-name match can be flagged as a softer duplicate for review.
- Import results distinguish inserted, duplicate and rejected rows; users can choose reject or skip behavior.

Lead score and intelligence

The qualification score is a deterministic, rule-based prioritization heuristic. The system stores a score and factor breakdown so a user can see how it was calculated. It is **not** a machine-learning conversion probability and should not be described as predicting which prospect will buy. Company insights are assembled from the stored lead information and the project?s analysis rules.

Outreach generation and safety of interpretation

- If an OpenAI-compatible model provider is configured, the backend can request a personalized first-touch email draft.
- If no provider key is set, it creates an offline mock draft, visibly labeled as mock output (not real AI generation).
- Generated email content is stored with status **draft**. The project does not send email automatically.

6. How to run and demonstrate the project

The repository includes a PowerShell helper script for setup and common commands. Run these commands from the project folder in Windows PowerShell.

Initial setup (first run)

```
cd "C:\Users\Dell\Desktop\sales genie ai"
.\run.ps1 setup
.\run.ps1 migrate
```

Start the backend and UI in separate PowerShell terminals

```
Terminal 1 ? Backend API:
.\run.ps1 api

Terminal 2 ? Streamlit UI:
.\run.ps1 ui
```

Where to open the app

URL	What it is
http://127.0.0.1:8501	Streamlit user interface (the app the presenter interacts with).
http://127.0.0.1:8000/docs	Interactive FastAPI endpoint documentation.
http://127.0.0.1:8000/health	Backend health response.

Suggested presentation demo

- Open the dashboard and explain that the numbers come from the backend database.
- Go to Leads, import data/sample_leads.csv, and explain validation and duplicate reporting.
- Select a prospect in Intelligence and explain the visible score factors and the score disclaimer.
- Open Outreach, choose options, generate a draft, and point out whether it is a real provider result or a clearly labeled mock.
- Emphasize that a human reviews the draft and that no email is sent automatically.

Verification commands

```
.\run.ps1 test ? run the automated test suite
.\run.ps1 status ? show migration state
.\run.ps1 check ? tests, migrations and live API check
```

7. Project status, limits and future work

Implemented scope

- Milestone 1: lead management, CSV import, duplicate detection, company insights, qualification score and UI.
- Milestone 2 first slice: prompt templates, optional provider-backed outreach with a labeled mock fallback, and draft storage.
- The project README reports 66 automated tests for the documented milestone implementation. Run the test command locally to confirm the current checkout.

Not included in the current release

- CRM synchronization/integration.
- Conversation intelligence and automated call/email analysis.
- A full analytics dashboard beyond the current summary counters and basic charts.
- A learned conversion model trained on real historical outcomes.
- Sending outreach emails automatically.

Limitations to explain honestly

- A rule-based score supports prioritization but does not guarantee lead quality or predict conversion.
- Company analysis depends on the available stored lead data; missing fields appear as information gaps.
- Real model-backed outreach requires a configured provider key and network access; mock drafts are not AI output.
- SQLite is intended for easy local use. PostgreSQL support is available, but the project?s documented test suite runs on SQLite.

A short explanation you can say

?SalesGenie AI is a Python-based sales support tool. Streamlit provides the browser interface, and FastAPI provides the backend REST API. The backend stores prospect records in SQLite by default and can also use PostgreSQL. Users can import and manage leads, view an explainable rule-based qualification score, and prepare an outreach email draft. Real language-model generation is optional; if no provider is configured, the draft is clearly marked as a mock. The current system does not send emails or include CRM synchronization.?

Project documents used

README.md; docs/TECH_STACK.md; docs/API_CONTRACTS.md; docs/ER_DIAGRAM.md; backend/app/main.py; backend/app/config.py; backend/app/database.py; frontend/app.py; frontend/api_client.py; frontend/pages/leads.py; frontend/pages/intelligence.py; frontend/pages/outreach.py; backend service modules.

End of project overview